```
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo $HOME
/home/akankshya
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo "Hello"
Hello
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo HOME
HOME
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ printenv
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=LAPTOP-84O9I4VO
PWD=/mnt/c/Users/akank
LOGNAME=akankshya
```

```
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo $USER
akankshya
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo HOME
HOME
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo $HOME
/home/akankshya
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ echo $SHELL
/bin/bash
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$
```

```
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ python3
Python 3.12.3 (main, Aug 31 2026, 10:18:26) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ which python3
/usr/bin/python3
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ pwd
/mnt/c/Users/akank
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ vim akankshya.sh
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ chmod +x akankshya.sh
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ akankshya.sh
akankshya.sh: command not found
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ ./akankshya.sh
Hello
akankshya@LAPTOP-84O9I4VO:/mnt/c/Users/akank$ mv akankshya.sh devs/aka
nkshya.sh
```

Important environment variables

Variable	Purpose
`HOME`	Your home directory
`USER`	Current username
`SHELL`	Your default login shell
`PATH`	Directories searched for executable commands
`PWD`	Current working directory
`OLDPWD`	Previous working directory
`LANG`	Language and locale settings
`TERM`	Terminal type

Try these:

```bash
echo $USER
echo $HOME
echo $SHELL
echo $PWD
echo $PATH
```

Remember: `$` is used to access the value of a variable.

What if you close the terminal?

If you close the terminal and open it again, this ordinary variable will generally disappear.

Why?

Because you created it only in the current shell session.

Difference between a shell variable and an environment variable

A shell variable belongs to your current shell.

An environment variable is exported so that child processes can access it too.

Use `export`:

```bash
export NAME="Akankshya"
```

Now it becomes an environment variable.

Check it:

```bash
printenv NAME
```

Output:

```plain text
Akankshya
```

How to remove a variable?

```bash
unset NAME
```

This removes the variable from your current shell.

---

3. What is PATH?

Very important for DevOps

Let's understand this with a real-life example.

Suppose you want to run Python.

You type:

```bash
python3
```

But how does Linux know where the Python program is located?

Does it search your entire computer?

No!

Linux checks specific directories listed in an environment variable called `PATH`.

PATH is a list of directories where Linux searches for executable commands.

Visualise how PATH works

You type a command

↓

Linux checks PATH

Searches the listed directories

↓

Example search

`/usr/local/bin`	Not found
`/usr/bin`	Found!
`/bin`	Not checked

↓

Command executes

4. How to add a directory to PATH

Suppose you create your own scripts inside:

```
Plain text
/home/akankshya/my-scripts
```

You want to execute those scripts from anywhere without typing their full path.

You can add this directory to PATH.

Use:

```
Bash
export PATH="$PATH:/home/akankshya/my-scripts"
```

Let's understand:

- `PATH` = Existing directories.
- `:` = Separator.
- `/home/akankshya/my-scripts` = New directory.
- `$PATH` = Keep all existing directories.

Important: This change lasts only for the current shell session unless you save it in a startup configuration file.

5. What is .bashrc?

This is another very important concept.

`.bashrc` is a configuration file used by Bash whenever an interactive non-login Bash shell starts.

It stores settings and commands that you want Bash to load automatically.

For example, imagine you always want:

```Bash
export NAME="Akankshya"
```

to be available whenever you open your terminal.

Instead of typing it every time, you can put it inside `.bashrc`.

Where is .bashrc located?

Usually:

```Plain text
/home/akankshya/.bashrc
```

Notice the dot before `bashrc`.

Files beginning with `.` are hidden files in Linux.

How to see hidden files?

```bash
ls -a
```

You will see files such as:

```
.bashrc
.profile
.bash_logout
```

How to open .bashrc?

Use:

```bash
nano ~/.bashrc
```

Or:

```bash
vim ~/.bashrc
```

Add your own environment variable

Go to the bottom of `.bashrc` and add:

```bash
export MY_NAME="Akankshya"
```

Save the file in nano:

1. Press `Ctrl + O`.
2. Press Enter.
3. Press `Ctrl + X`.

How to apply changes immediately?

Run:

```bash
source ~/.bashrc
```

Now check:

```bash
echo $MY_NAME
```

Output:

Plain text

```
Akankshya
```

You don't need to close and reopen the terminal!

Command	Purpose
`env`	Display environment variables
`printenv`	Display environment variables
`echo $HOME`	Display home directory
`echo $PATH`	Display PATH
`export NAME=value`	Create/export environment variabl
`unset NAME`	Remove variable
`which python3`	Find executable location
`type python3`	Identify command type
`ls -a`	Show hidden files
`nano ~/.bashrc`	Edit Bash configuration
`source ~/.bashrc`	Reload Bash configuration
`pwd`	Display current directory

1. What are archives and compression?

First, understand that these are two different concepts.

Archiving: Combining multiple files and folders into one single file.

Compression: Reducing the size of files to save storage space and make transfers faster.

Imagine you have these files:

```
Plain text
linux-practice/
    notes.txt
    1.txt
    akankshya.txt
```

You want to send all three files to someone.

Instead of sending them separately, you can combine them into one archive.

Before archiving

notes.txt

1.txt

akankshya.txt

↓

Archive them

myfiles.tar

One archive containing all three files

2. Understanding the tar command

`tar` stands for Tape Archive.

It is used to combine multiple files and directories into a single archive.

Basic syntax

```bash
tar [options] archive_name files
```

Let's understand the important options.

Option	Meaning
`-c`	Create an archive
`-v`	Verbose (show files being processed)
`-f`	Specify archive filename
`-x`	Extract an archive
`-t`	List archive contents
`-z`	Use gzip compression
`-j`	Use bzip2 compression
`-J`	Use xz compression

Remember these options. You'll use them frequently.

3. Create your first archive

Let's practise.

Step 1: Go to your practice directory

```bash
cd ~/linux-practice
```

Check your files:

```bash
ls
```

Step 2: Create an archive

```bash
tar -cvf myfiles.tar notes.txt 1.txt akankshya.txt
```

Let's break it down:

- `tar` = Archive command.
- `-c` = Create.
- `-v` = Show the files being added.
- `-f` = Specify the archive filename.
- `myfiles.tar` = Name of the archive.
- Remaining names = Files to include.

Step 3: Verify

```bash
ls
```

You should now see:

```
1.txt
akankshya.txt
myfiles.tar
notes.txt
my/
```

Your first archive created!

4. How to view archive contents

Suppose you want to see which files are inside `myfiles.tar` without extracting them.

Use:

```bash
tar -tvf myfiles.tar
```

Here:

- `-t` = List contents.
- `-v` = Show details.
- `-f` = Specify archive.

This displays the files stored inside the archive.

5. How to extract an archive

Suppose you received `myfiles.tar` from someone and want to recover the original files.

Use:

```bash
tar -xvf myfiles.tar
```

Breaking it down:

- `-x` = Extract.
- `-v` = Show extracted files.
- `-f` = Specify archive.

The files will be extracted into your current directory, potentially replacing existing files with the same names.

Extract into a different folder

First create a directory:

```bash
mkdir extracted
```

Now extract:

```bash
tar -xvf myfiles.tar -C extracted
```

`-C` tells tar which directory to extract into.

Check:

Bash

```
ls extracted
```